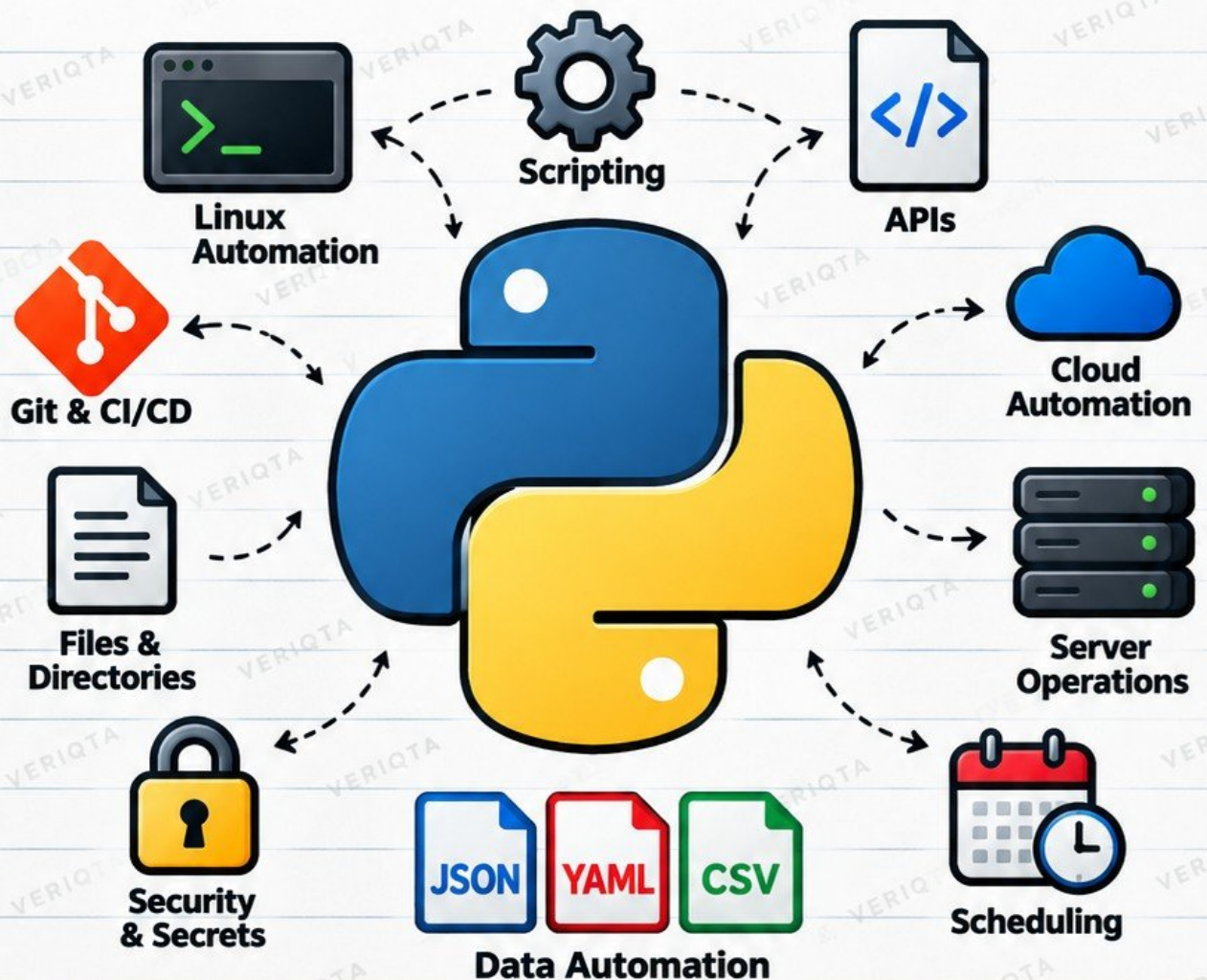


Python

Automation

Pocket Guide



**Practical Python for DevOps, Cloud, Linux
and Infrastructure Automation**

01

Python Automation: What Are We Automating?

1. What is Automation?

Automation means using code to do **repetitive tasks** automatically instead of doing them **manually**.



2. Automation vs Scripting

Automation	Scripting
Usually larger and more structured	Usually small and simple
Often runs on a schedule	Run manually when needed
Can include error handling, logging and monitoring	May not include these
Used for continuous operational tasks	Used for quick one-off tasks

3. Repetitive Operational Work

In infrastructure and DevOps, many tasks are repeated regularly. Examples:

- Checking server health
- Backing up files
- Cleaning old logs
- Managing users and permissions
- Starting or stopping services
- Collecting system information
- Deploying applications



4. Where Python Fits in DevOps

Python is widely used in DevOps because it is:

- ✓ Easy to learn
- ✓ Powerful and flexible
- ✓ Available on Linux, macOS and Windows
- ✓ Large library ecosystem
- ✓ Great for system administration, APIs, cloud platforms and automation

5. Examples of Useful Automation



Analyze Logs
Find errors or failed logins



File Management
Move, backup or clean files



Cloud Automation
Create or manage cloud resources



Service Management
Start, stop or restart services



Notifications
Send alerts on failure or success



Reports
Generate daily or weekly reports

6. Simple Example

This Python script checks disk usage and prints a warning if usage is above 80%.

```
import shutil

usage = shutil.disk_usage("/")
percent = (usage.used / usage.total) * 100

if percent > 80:
    print(f"Warning: Disk usage is {percent:.1f}%")
else:
    print(f"Disk usage is {percent:.1f}%")
```

7. When Not to Automate

- ✗ Tasks that are done rarely
- ✗ One-time manual tasks
- ✗ Tasks that require human judgment
- ✗ Tasks that change frequently
- ✗ Very simple tasks that take only a few seconds

8. Key Takeaways

- ✓ Automation saves time and reduces human error
- ✓ Python is a great language for DevOps automation
- ✓ Focus on real operational problems
- ✓ Start with small useful scripts and improve over time
- ✓ Do not automate everything



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1

Your Python Automation Environment

1. What You Need

To write and run Python automation scripts, you need:

- ✓ Python 3 (use `python3`)
- ✓ `pip` (Python package manager)
- ✓ Virtual environment (keeps projects isolated)
- ✓ A code editor (VS Code, any editor)
- ✓ A terminal or command line
- ✓ A clean project structure



2. Check Python and pip Versions

First, confirm Python and pip are installed.

```
$ python3 --version
Python 3.11.6

$ pip3 --version
pip 23.2.1 from /usr/lib/python3/dist-packages/pip
(pyenv 3.11)
```

3. Python REPL (Interactive Shell)

The REPL lets you test Python commands quickly.

```
$ python3
Python 3.11.6 (main, Oct 2 2023, 12:00:00)
Type "help", "copyright", "credits" or "exit()."
>>> 2 + 3
5
>>> print("Hello VERIQTA")
Hello VERIQTA
>>> exit()
```

4. Virtual Environment

A virtual environment keeps your project packages separate.

```
$ python3 -m venv venv # create virtual environment
$ source venv/bin/activate # activate it (Linux/Mac)
(venv) $ pip install requests # install packages
(venv) $ pip list # see installed packages
(venv) $ deactivate # exit the virtual environment
```

5. Python Files (.py) and Running Scripts

Create a Python file with your code.

```
hello.py

print("Hello, VERIQTA!")
name = "DevOps"
print(f"Welcome, {name}!")
```

Run the script:

```
$ python3 hello.py
Hello, VERIQTA!
Welcome, DevOps!
```

6. Project Structure

Keep your project organized from the start.

```
python-automation/
├── venv/ # virtual environment
├── scripts/
│   └── hello.py # your scripts
├── requirements.txt # project packages
└── README.md # project notes
```

7. Installing Packages with pip

Use `pip` to install Python packages for automation tasks.

```
(venv) $ pip install requests
(venv) $ pip install boto3
(venv) $ pip freeze > requirements.txt
# save packages for later
```

8. Common Commands

<code>python3 --version</code>	Check Python version
<code>pip3 --version</code>	Check pip version
<code>python3</code>	Open Python REPL
<code>pip install <package></code>	Install a package
<code>pip list</code>	Show installed packages
<code>pip freeze</code>	List packages (for file)
<code>python3 script.py</code>	Run a Python script

9. Key Takeaways

- ✓ Use Python 3 and `python3`
- ✓ Always use a virtual environment
- ✓ Use `pip` to install packages
- ✓ Test commands in the REPL
- ✓ Keep a clean project structure
- ✓ Use `.py` files for your scripts
- ✓ Check versions and dependencies
- ✓ This environment is the foundation for all your automation work



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

03

Variables, Data Types & Operators

Strings, Integers, Floats, Booleans, None, Assignment, Comparison, Arithmetic, Type Conversion and Practical Infrastructure Examples

1. What are Variables?

A variable is a named container that holds a value. You can store data and use it later in your script.

```
server = "web-01"
port = 8080
is_running = True
```



2. Common Python Data Types

Data Type	Example	Description	Common Use
str	"web-01"	Text (string)	Hostnames, paths, logs, messages
int	100	Whole numbers	Ports, counts, IDs
float	3.14	Decimal numbers	Metrics, size, cost
bool	True / False	Boolean value	Conditions, checks
None	None	Represents no value	Missing data, optional values

3. Assignment and Reassignment

Use = to assign a value to a variable.

```
name = "VERIQTA"
count = 5
count = count + 1 # now count is 6
name = "DevOps" # reassign value
```

4. Type Conversion

Convert between different data types.

```
port_str = "8080"
port_int = int(port_str) # string to integer
cpu = 75
cpu_str = str(cpu) # integer to string
memory = float("16.5") # string to float
is_active = bool(1) # 1 is True, 0 is False
```

5. Arithmetic Operators

Used for mathematical calculations.

Operator	Example	Result	Use Case
+	5 + 3	8	Add values
-	5 - 3	2	Subtract values
*	5 * 3	15	Multiply values
/	10 / 3	3.33	Division (float)
//	10 // 3	3	Division (integer)
%	10 % 3	1	Remainder
**	2 ** 3	8	Power (exponent)

6. Comparison Operators

Compare values. The result is True or False.

Operator	Example	Result	Use Case
==	5 == 5	True	Equal to
!=	5 != 3	True	Not equal to
>	5 > 3	True	Greater than
<	5 < 3	False	Less than
>=	5 >= 5	True	Greater or equal
<=	5 <= 3	False	Less or equal

7. Logical Operators

Combine multiple conditions.

Operator	Example	Result	Use Case
and	True and False	False	Both conditions True
or	True or False	True	At least one True
not	not True	False	Reverse the result

8. Practical Infrastructure Examples

Using variables and operators in real DevOps tasks.

```
host = "web-01"
port = 22
cpu_usage = 85
threshold = 80
is_high = cpu_usage > threshold
message = f"{host} CPU is high: {cpu_usage}%"
print(message) # web-01 CPU is high: 85%
```



9. Key Takeaways

- Variables store data for use in your script
- Python has str, int, float, bool and None
- Use operators to perform calculations and comparisons
- Use type conversion when working with different data
- These basics are used in every real automation script

10. Common Beginner Mistakes

- Using a variable before assigning a value
- Confusing = (assign) with == (compare)
- Treating a number as a string
- Forgetting to convert types when reading input
- Using the wrong operator for the condition

Strings for Real Automation Work

String methods, f-strings, splitting, joining, stripping, searching, replacing, parsing command output, paths, hostnames and log text.

1. Creating and Using Strings

Strings store text such as hostnames, paths, logs and messages.

```
name = "VERIQTA"
message = 'Automation with Python'
print(name)
print(message)
```



2. f-Strings (Formatted Strings)

Use f-strings to include variables in a string.

```
host = "web-01"
port = 22
msg = f"Connecting to {host} on port {port}"
print(msg) # Connecting to web-01 on port 22
```



3. Common String Methods

Useful methods for automation tasks.

Method	Example	Result	Use Case
lower()	"WEB".lower()	"web"	Case insensitive check
upper()	"web".upper()	"WEB"	Standardize text
strip()	" web ".strip()	"web"	Remove spaces/newlines
replace()	"a,b".replace(",",";")	"a;b"	Replace text
startswith()	"web-01".startswith("web")	True	Check prefix
endswith()	"file.log".endswith(".log")	True	Check suffix
find()	"error".find("or")	2	Find position
count()	"a,b,a".count("a")	2	Count occurrences

4. Split and Join Strings

Split a string into a list or join a list into a string.

```
servers = "web-01,web-02,web-03"
server_list = servers.split(",")
print(server_list)
# ['web-01', 'web-02', 'web-03']

joined = ",".join(server_list)
print(joined) # web-01,web-02,web-03
```



5. Searching and Replacing

Find and replace text in strings.

```
log_line = "ERROR: Failed to start service"
if "ERROR" in log_line:
    print("Error found in log")
new_line = log_line.replace("ERROR", "WARN")
print(new_line) # WARN: Failed to start service
```



6. Parsing Command Output

Strings are often used to parse Linux command output.

```
output = """Filesystem
Size Used\ dev/sda1 50G 20G
/dev/sdb1 100G 10G """
lines = output.strip().split("\n")
for line in lines[1:]: # skip header
    parts = line.split()
    fs, size, used = parts
    print(f"{fs} - {used}/{size}")
```



7. Working with Paths and Hostnames

Manipulate file paths and extract hostnames.

```
path = "/var/log/nginx/access.log"
filename = path.split("/")[-1]
print(filename) # access.log

hostname = "web-01.prod.example.com"
short_name = hostname.split(".")[0]
print(short_name) # web-01
```



8. Working with Log Text

Extract useful information from log lines.

```
log = "2024-01-01 INFO User admin logged in"
parts = log.split()
date = parts[0]
level = parts[1]
user = parts[3]
print(f"Date: {date}")
print(f"Level: {level}")
print(f"User: {user}")
```



9. Practical Infrastructure Examples

- ✓ Extract hostnames from a server list
- ✓ Parse log files and find errors
- ✓ Build file paths for backups
- ✓ Clean and validate user input
- ✓ Process command output from Linux tools
- ✓ Generate standardized names and messages

10. Key Takeaways

- ✓ Strings help you work with real infrastructure data
- ✓ Use f-strings for readable and dynamic text
- ✓ Learn common string methods
- ✓ Split and join strings when processing lists
- ✓ Parse command output and log files
- ✓ Use strings to handle paths, hostnames and configuration data



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

05

Lists, Dictionaries, Tuples & Sets

Choosing the correct collection, looping through servers, storing configuration, removing duplicates, and representing structured operational data.

1. What Are These Collections?

Different collections store data in different ways.

Collection	Syntax Example	Key Point	Common Use
List	<code>["web", "db"]</code>	Ordered, mutable	Servers, items, results
Dictionary	<code>{"host": "web-01"}</code>	Key-value pairs	Configuration, structured data
Tuple	<code>("web", 80)</code>	Ordered, immutable	Fixed data, constants
Set	<code>{"web", "db"}</code>	Unique values	Remove duplicates, unique hosts

2. Lists: Ordered and Mutable

Use lists to store multiple items in order.

```
servers = ["web-01", "web-02", "db-01"]
servers.append("app-01")
print(servers[0]) # first item
print(len(servers)) # number of items
```



Looping through a list:

```
for server in servers:
    print(f"Checking {server}")
```

3. Dictionaries: Key-Value Pairs

Use dictionaries to store related information.

```
server = {
    "name": "web-01",
    "ip": "10.0.0.10",
    "role": "web",
    "port": 80
}
print(server["name"])
print(server.get("env", "not set"))
```

Real World Example

Store server configuration:

```
{
  "name": "web-01",
  "ip": "10.0.0.10",
  "role": "web",
  "env": "prod"
}
```

4. Tuples: Ordered and Immutable

Use tuples for fixed data that should not change.

```
server = ("web-01",
          "10.0.0.10", 80)
name, ip, port = server
print(name)
print(ip)
print(port)
```

When to Use

- ✓ Fixed configuration
- ✓ Function return values
- ✓ Data that should not be modified
- ✓ Coordinates, constants

5. Sets: Unique Values

Use sets to store unique items and remove duplicates.

```
hosts = ["web-01", "web-02", "web-01", "db-01"]
unique_hosts = set(hosts)
print(unique_hosts) # duplicates removed
unique_hosts.add("app-01")
print(unique_hosts)
```

Set Operations

```
a = {1, 2, 3}
b = {3, 4}
print(a | b) # union
print(a & b) # intersection
print(a - b) # difference
```

6. Choosing the Right Collection

Use the right collection based on your need.

Need	Use	Example
Ordered list of items	List	<code>["web", "db"]</code>
Key-value configuration	Dictionary	<code>{"host": "web-01"}</code>
Fixed data (no changes)	Tuple	<code>("web", 80)</code>
Unique items / remove duplicates	Set	<code>{"web", "db"}</code>

7. Practical Infrastructure Example

Store and process a list of servers with details.

```
servers = [
    {"name": "web-01", "ip": "10.0.0.10"},
    {"name": "web-02", "ip": "10.0.0.11"},
    {"name": "db-01", "ip": "10.0.1.20"}
]
for server in servers:
    print(f"Checking {server['name']} at {server['ip']}")
```



8. Working with Configuration Data

Using a dictionary to store application configuration.

```
config = {
    "app": "nginx",
    "port": 80,
    "env": "prod",
    "servers": ["web-01", "web-02"]
}
print(config["app"])
print(config["servers"])
```



9. Key Takeaways

- ✓ Lists store ordered items and can be changed
- ✓ Dictionaries store key-value pairs
- ✓ Tuples are ordered and cannot be changed
- ✓ Sets keep unique items and remove duplicates
- ✓ Choose the right collection based on your use case
- ✓ These collections are essential for real automation work

10. Common Beginner Mistakes

- ✗ Using a list when a dictionary is better
- ✗ Forgetting that tuples cannot be changed
- ✗ Expecting sets to keep items in order
- ✗ Using duplicate values when a set should be used
- ✗ Accessing a dictionary key that does not exist
- ✗ Confusing list methods such as `append()` and `extend()`



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

5

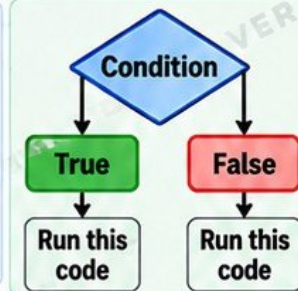
Conditions & Decision Logic

if, elif, else, comparison and logical operators, membership tests, exit-status decisions, health checks, and conditional automation.

1. if, elif, else (Decision Making)

Run different code based on a condition.

```
cpu_usage = 85
if cpu_usage > 80:
    print("CPU is high")
elif cpu_usage > 60:
    print("CPU is moderate")
else:
    print("CPU is normal")
```



2. Comparison Operators

Compare values. The result is True or False.

Operator	Example	Result	Meaning
==	5 == 5	True	Equal to
!=	5 != 3	True	Not equal to
>	5 > 3	True	Greater than
<	5 < 3	False	Less than
>=	5 >= 5	True	Greater or equal
<=	3 <= 2	False	Less or equal

3. Logical Operators

Combine multiple conditions.

Operator	Example	Result	Meaning
and	True and False	False	Both must be True
or	True or False	True	At least one True
not	not True	False	Reverse the result

4. Membership Tests

Check if a value exists in a collection.

```
servers = ["web-01", "web-02", "db-01"]
if "web-01" in servers:
    print("web-01 found")
if "cache-01" not in servers:
    print("cache-01 not found")
```



5. Exit-Status Decisions

Use the return code of a Linux command.

```
import subprocess
result = subprocess.run(["ping", "-c", "1", "8.8.8.8"])

if result.returncode == 0:
    print("Host is reachable")
else:
    print("Host is unreachable")
```

Return Codes

0	Success
1+	Failure

6. Practical Health Check Example

Check if a website is responding.

```
import requests
url = "https://example.com"
try:
    response = requests.get(url, timeout=5)
    if response.status_code == 200:
        print("Website is up")
    else:
        print(f"Website returned {response.status_code}")
except:
    print("Website is not reachable")
```



7. Real DevOps Automation Example

Restart a service if it is not running.

```
import subprocess
service = "nginx"
result = subprocess.run(["systemctl", "is-active", service],
                        capture_output=True, text=True)

if "active" in result.stdout:
    print(f"{service} is running")
else:
    print(f"{service} is not running, starting...")
    subprocess.run(["sudo", "systemctl", "start", service])
```



8. Key Takeaways

- ✓ Use if, elif, else to make decisions
- ✓ Comparison and logical operators help you build conditions
- ✓ Use in and not in to check membership
- ✓ Use exit codes to detect success or failure
- ✓ Apply conditions in health checks and automation scripts

9. Common Beginner Mistakes

- ✗ Using = instead of == for comparison
- ✗ Forgetting to handle the else case
- ✗ Using and/or incorrectly in conditions
- ✗ Checking for the wrong value or type
- ✗ Not handling command failures or exceptions

Loops: Automating Repetitive Work

for, while, range(), break, continue, nested loops, and safely processing collections of files, hosts, or services.

1. for Loop (Iterating Through a List)

Use for loop to process each item in a collection.

```
servers = ["web-01", "web-02", "db-01"]
for server in servers:
    print(f"Processing {server}")
```



Output:

```
Processing web-01
Processing web-02
Processing db-01
```

2. range() Loop (Repeating a Task)

Use range() to repeat a task a specific number of times.

```
for i in range(1, 6):
    print(f"Backup attempt {i}")
```

Output:

```
Backup attempt 1
Backup attempt 2
Backup attempt 3
Backup attempt 4
Backup attempt 5
```

range() Examples

range(5)	0, 1, 2, 3, 4
range(1, 6)	1, 2, 3, 4, 5
range(0, 10, 2)	0, 2, 4, 6, 8



3. while Loop (Repeat Until Condition)

Use while loop when you do not know the exact number of iterations.

```
count = 0
while count < 3:
    print(f"Health check (count + 1)")
    count += 1
```



Output:

```
Health check 1
Health check 2
Health check 3
```

4. break and continue

Use break to exit a loop and continue to skip an iteration.

Using break

```
for i in range(1, 6):
    if i == 3:
        break # stop when i is 3
    print(i)
```

Output:

```
1
2
```

Using continue

```
for i in range(1, 6):
    if i == 3:
        continue # skip 3
    print(i)
```

Output:

```
1
2
4
5
```

5. Nested Loops

Use nested loops to handle multiple collections.

```
environments = ["dev", "staging", "prod"]
servers = ["web-01", "web-02"]
for env in environments:
    for server in servers:
        print(f"{env}: {server}")
```



Output: dev: web-01-01 staging: web-02 prod: web-01
dev: web-02 staging: web-02 prod: web-02

6. Looping Through Files

Use loops to process files in a directory.

```
import os
for file in os.listdir("/var/log"):
    if file.endswith(".log"):
        print(f"Log file: {file}")
```



7. Practical Automation Example (Check Multiple Services)

Loop through services and check if they are running.

```
import subprocess
services = ["nginx", "docker", "ssh"]
for service in services:
    result = subprocess.run(["systemctl", "is-active", service],
                           capture_output=True, text=True)
    if "active" in result.stdout:
        print(f"{service} is running")
    else:
        print(f"{service} is NOT running")
```

Output (example):
nginx is running
docker is running
ssh is NOT running



8. Looping Through Hosts (From a File)

Read a list of hosts and run a command on each.

```
with open("hosts.txt") as f:
    for host in f:
        host = host.strip()
        if not host or host.startswith("#"):
            continue # skip empty lines/comments
        print(f"Checking {host}")
```



9. Key Takeaways

- ✓ Use for, while and range() for repetitive work
- ✓ Use break to exit a loop and continue to skip iterations
- ✓ Nested loops help you process multiple collections
- ✓ Loops are useful for files, hosts, services and real automation tasks
- ✓ Always add checks to avoid processing unwanted items

10. Common Beginner Mistakes

- ✗ Forgetting to update the loop condition in while loops
- ✗ Using break when you meant continue (or vice versa)
- ✗ Not handling empty items when looping through files or hosts
- ✗ Causing infinite loops
- ✗ Not validating data inside the loop



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

7

Functions: Building Reusable Automation

Defining functions, parameters, return values, scope, type hints, docstrings, separating responsibilities, and avoiding one giant script.

1. What are Functions?

Functions are reusable blocks of code that perform a specific task.

```
def greet():
    print("Hello from a function!")

greet() # call the function
```



Benefits

- ✓ Reusable code
- ✓ Cleaner scripts
- ✓ Easier to maintain
- ✓ Reduce repetition

3. Return Values

Functions can return values that you can use later.

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result) # 8
```

Key Point

Use **return** to send a value back from a function.



2. Function Parameters

Pass values to functions using parameters.

```
def check_cpu(host, threshold):
    print(f"Checking {host}")
    print(f"Threshold: {threshold}%")

check_cpu("web-01", 80) # arguments
```



4. Scope (Local vs Global)

Variables inside a function are local by default.

```
x = 10 # global variable

def show_value():
    x = 5 # local variable
    print("Inside function:", x)

show_value()
print("Outside function:", x)
```

Output

Inside function: 5
Outside function: 10

Key Point

Local variables exist only inside the function. Use global only when necessary.

5. Type Hints (Optional but Recommended)

Type hints make your code easier to understand.

```
def calculate_disk_usage(path: str) -> float:
    """Return disk usage percentage for a path."""
    import shutil
    total, used, free = shutil.disk_usage(path)
    return (used / total) * 100
```

Common Types

```
str # text
int # integers
float # decimals
bool # True/False
list # lists
dict # dictionaries
```

7. Separating Responsibilities

Break your script into small, focused functions.

```
def get_servers():
    return ["web-01", "web-02", "db-01"]

def check_server(host: str) -> bool:
    # simulate a health check
    print(f"Checking {host}")
    return True

def main():
    servers = get_servers()
    for server in servers:
        check_server(server)
if __name__ == "__main__":
    main()
```

Why Separate?

- ✓ Easier to read
- ✓ Easier to test
- ✓ Reusable functions
- ✓ Simpler debugging
- ✓ Avoids one giant script



6. Docstrings (Documentation)

Use docstrings to explain what your function does.

```
def restart_service(service: str) -> bool:
    """
    Restart a system service.
    Args:
        service (str): name of the service.
    Returns:
        bool: True if restart was successful.
    """
    import subprocess
    result = subprocess.run(["systemctl", "restart", service])
    return result.returncode == 0
```

8. Real Automation Example

Function to check if a server is reachable and return the result.

```
import subprocess

def is_server_up(host: str, timeout: int = 5) -> bool:
    """Check if a server is reachable using ping."""
    result = subprocess.run(["ping", "-c", "1", host],
                            stdout=subprocess.DEVNULL,
                            stderr=subprocess.DEVNULL,
                            timeout=timeout)
    return result.returncode == 0
```



9. Key Takeaways

- ✓ Functions help you write reusable and modular code
- ✓ Use parameters to pass values and return values to get results
- ✓ Understand local and global scope
- ✓ Use type hints and docstrings for better, readable code
- ✓ Separate responsibilities and avoid one giant script
- ✓ Functions make your automation scripts easier to maintain and scale

10. Common Beginner Mistakes

- ✗ Writing one giant script instead of using functions
- ✗ Forgetting to call the function
- ✗ Not using return when you need a result
- ✗ Confusing local and global scope
- ✗ Not using parameters (hardcoding values)
- ✗ Skipping docstrings and type hints
- ✗ Making functions too long (do one thing only)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Files, Directories & Paths

pathlib, reading and writing files, creating directories, checking existence, copying and moving files, and safe filesystem automation.

1. Working with Paths (pathlib)

Use pathlib to work with file and directory paths.

```
from pathlib import Path
# create a path object
path = Path("/var/log/nginx")
print(path)           # /var/log/nginx
print(path.name)       # nginx
print(path.parent)     # /var/log
print(path.exists())   # True or False
```



2. Checking Existence and File Information

Check if files or directories exist and get information.

```
from pathlib import Path
path = Path("/etc/hosts")
print(path.exists())   # True or False
print(path.is_file())  # True if file
print(path.is_dir())   # True if directory
print(path.stat().st_size) # file size in bytes
print(path.stat().st_mtime) # last modified time
```



3. Creating Directories

Create new directories, including parent directories.

```
from pathlib import Path
new_dir = Path("automation/reports")
new_dir.mkdir(parents=True, exist_ok=True)
print(f"Directory created: {new_dir}")
```

Parameters

- ✓ **parents=True** creates all missing parent directories.
- ✓ **exist_ok=True** does not fail if the directory already exists.

4. Reading Files

Read the contents of a file.

```
from pathlib import Path
file_path = Path("/etc/hosts")
# Read entire file
content = file_path.read_text()
print(content)
# Read line by line
with file_path.open() as f:
    for line in f:
        print(line.strip())
```

Common Methods

- ✓ **read_text()**
- ✓ **read_bytes()**
- ✓ **read_lines()**
- ✓ **open()**
- ✓ **encoding="utf-8"**



5. Writing Files

Write text or data to a file.

```
from pathlib import Path
file_path = Path("automation/output.txt")
# Write text (overwrites if exists)
file_path.write_text("Automation log\n")
# Append text
with file_path.open("a") as f:
    f.write("New log entry\n")
```

Write Options

- ✓ **write_text()**
- ✓ **write_bytes()**
- ✓ **open("w")** overwrite
- ✓ **open("a")** append
- ✓ **Specify encoding**
- ✓ **Handle exceptions**

6. Copying and Moving Files

Copy or move files and directories.

```
from pathlib import Path
import shutil
src = Path("reports/app.log")
dst = Path("backup/app.log")
# Copy a file
shutil.copy2(src, dst) # preserves metadata
# Move a file
shutil.move(src, dst)
# Copy a directory
shutil.copytree("reports", "reports_backup")
```



7. Practical Filesystem Automation Example

Process all log files in a directory.

```
from pathlib import Path
log_dir = Path("/var/log")
output_dir = Path("automation/processed")
output_dir.mkdir(parents=True, exist_ok=True)

for log_file in log_dir.glob("*.log"):
    print(f"Processing {log_file.name}")
    content = log_file.read_text()
    new_file = output_dir / log_file.name
    new_file.write_text(content)
```

```
> _
Output:
Processing syslog
Processing auth.log
Processing nginx.log
Processing app.log
```



8. Useful Path Methods

Common pathlib methods for automation.

Method	Example	Description
exists()	<code>path.exists()</code>	Check if path exists
is_file()	<code>path.is_file()</code>	True if file
is_dir()	<code>path.is_dir()</code>	True if directory
glob()	<code>path.glob("*.log")</code>	Find matching files
mkdir()	<code>path.mkdir()</code>	Create directory
read_text()	<code>path.read_text()</code>	Read file as text
write_text()	<code>path.write_text()</code>	Write text to file
rename()	<code>path.rename(new_path)</code>	Rename or move file
rmdir()	<code>path.rmdir()</code>	Remove empty directory
unlink()	<code>path.unlink()</code>	Delete a file

9. Key Takeaways

- ✓ Use **pathlib** for safe and modern path handling
- ✓ Create directories with **parents=True** when needed
- ✓ Check if files or directories exist **before** using them
- ✓ Use **read_text()** and **write_text()** for easy file operations
- ✓ Use **shutil** for copying and moving files and directories
- ✓ Always handle exceptions when working with files

10. Common Beginner Mistakes

- ✗ Using string paths instead of **pathlib**
- ✗ Not checking if a file or directory exists
- ✗ Forgetting **parents=True** when creating nested directories
- ✗ Overwriting files accidentally
- ✗ Not handling file permissions or exceptions
- ✗ Using **shutil.copy()** instead of **shutil.copy2()** when metadata matters



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Working with JSON, YAML & CSV

Reading and writing structured data, parsing configuration, API responses, inventory files, CSV reports, serialization, and validation basics.

1. Overview: When to Use What

Different formats are used for different purposes.

Format	File Extension	Common Use	Example
JSON	.json	API responses, configuration, structured data	<code>{ "key": "value" }</code>
YAML	.yaml / .yml	Configuration files, Kubernetes, Ansible inventory	<code>key: value</code>
CSV	.csv	Tabular data, reports, logs, exported data	<code>name,age,role web-01,30,devops</code>

2. Working with JSON

Read and write JSON data using the `json` module.

```
import json
# Read JSON file
with open("servers.json", "r") as f:
    data = json.load(f)
    print(data["servers"][0]["name"])

# Write JSON file
output = {"status": "success",
          "count": 2}
with open("output.json", "w") as f:
    json.dump(output, f, indent=4)
```

Example: servers.json

```
{
  "servers": [
    {
      "name": "web-01",
      "ip": "10.0.0.10",
    },
    {
      "name": "web-02",
      "ip": "10.0.0.11"
    }
  ]
}
```

JSON

3. Working with YAML

Use PyYAML to read and write YAML files.

```
import yaml
# Read YAML file
with open("config.yaml", "r") as f:
    config = yaml.safe_load(f)
    print(config["app"]["port"])

# Write YAML file
new_config = {"app": {"port": 8080,
                     "env": "prod"}}
with open("output.yaml", "w") as f:
    yaml.safe_dump(new_config, f, sort_keys=False)
```

Example: config.yaml

```
app:
  name: nginx
  port: 80
  env: dev
servers:
  - web-01
  - web-02
```

YAML

4. Working with CSV

Use the `csv` module to read and write CSV files.

```
import csv
# Read CSV file
with open("servers.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["ip"])

# Write CSV file
fieldnames = ["name", "ip", "status"]
with open("output.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({"name": "web-01", "ip": "10.0.0.10",
                    "status": "up"})
```

Example: servers.csv

```
name,ip,status
web-01,10.0.0.10,up
web-02,10.0.0.11,up
```

CSV

5. Parsing Real Data (API Response, Inventory, Logs)

JSON from an API, YAML inventory, and CSV report examples.

JSON API Response

```
import json
response = '{"service":"nginx",
            "status":"running"}'
data = json.loads(response)
print(data["service"]) # nginx
print(data["status"]) # running
```

YAML Inventory

```
import yaml
with open("hosts.yaml") as f:
    inventory = yaml.safe_load(f)
    for host in inventory["servers"]:
        print(host["name"], host["ip"])
```

CSV Report Analysis

```
import csv
with open("report.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if row["status"] == "down":
            print(f"{row['name']} is down")
```



8. Key Takeaways

- ✓ JSON is common for API responses and structured data
- ✓ YAML is widely used for configuration and inventories
- ✓ CSV is ideal for tabular data and reports
- ✓ Use proper modules: `json`, `yaml` (PyYAML), and `csv`
- ✓ Always validate data and handle errors
- ✓ These formats are essential for real automation work

6. Serialization and Deserialization

Convert Python objects to and from different formats.

Format	Serialize (Python → File)	Deserialize (File → Python)
JSON	<code>json.dump(obj, f)</code> <code>json.dumps(obj)</code>	<code>json.load(f)</code> <code>json.loads(s)</code>
YAML	<code>yaml.safe_dump(obj, f)</code>	<code>yaml.safe_load(f)</code>
CSV	<code>csv.DictWriter(...)</code>	<code>csv.DictReader(...)</code>

7. Validation Basics

Basic checks to ensure your data is valid.

JSON Validation (Check Required Keys)

```
data = json.load(open("config.json"))
required = ["app", "port"]
for key in required:
    if key not in data:
        raise ValueError(f"Missing key: {key}")
```

YAML Validation (Simple Check)

```
config = yaml.safe_load(open("config.yaml"))
if not isinstance(config, dict):
    raise ValueError("YAML must be a mapping")
if "app" not in config:
    raise ValueError("Missing 'app' section")
```

9. Common Beginner Mistakes

- ✗ Confusing JSON and YAML syntax
- ✗ Not using `safe_load()` for YAML
- ✗ Forgetting to close files or use with `open()`
- ✗ Assuming keys always exist
- ✗ Not handling missing or invalid data
- ✗ Using the wrong module for the format



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Errors, Exceptions & Defensive Automation

try, except, else, finally, raising exceptions, expected vs unexpected failures, useful error messages, cleanup, and fail-safe behavior.

1. Basic try, except, else, finally

Handle errors and always run cleanup code.

```
try:
    result = 10 / 2
    print(f"Result: {result}")
except ZeroDivisionError as e:
    print(f"Cannot divide by zero: {e}")
else:
    print("No errors occurred")
finally:
    print("This always runs (cleanup)")
```

Execution Flow

- ↓ 1. try (run code)
- ↓ 2. except (if error occurs)
- ↓ 3. else (if no error)
- ↓ 4. finally (always runs)

2. Handling Specific Exceptions

Catch different types of errors with clear messages.

```
try:
    with open("config.yaml", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("Config file not found. Please check the path.")
except PermissionError:
    print("Permission denied. Check file permissions.")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Common Exceptions

FileNotFoundError
File does not exist

PermissionError
No permission to access

ValueError
Invalid value

TypeError
Wrong data type

KeyError
Missing dictionary key

Exception
Catch all other errors

3. Raising Exceptions

Raise your own exceptions for validation and safety.

```
def validate_port(port: int) -> int:
    if port < 1 or port > 65535:
        raise ValueError(f"Invalid port: {port}. Must be between 1 and 65535.")
    return port

try:
    validate_port(70000)
except ValueError as e:
    print(e)
```

Why Raise Exceptions?

- ✓ Stop the script when input is invalid
- ✓ Prevent unexpected behavior
- ✓ Give clear and useful error messages
- ✓ Make automation more reliable

4. Expected vs Unexpected Failures

Handle expected errors and log unexpected ones.

```
import subprocess

try:
    result = subprocess.run(
        ["ping", "-c", "1", "example.com"],
        capture_output=True, text=True, check=True
    )
    print("Host is reachable")
except subprocess.CalledProcessError:
    print("Host is not reachable (expected failure)")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Examples

- ✓ **Expected Failures**
 - Host is down
 - File not found
 - Service not running
 - Invalid input
- Unexpected Failures**
 - Network issues
 - Permission errors
 - Bug in the script
 - Unknown exceptions

5. Useful Error Messages

Provide clear and actionable error messages.

```
def read_config(path: str) -> dict:
    try:
        with open(path, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        raise FileNotFoundError(
            f"Configuration file not found: {path}. Please check the path and try again."
        )
    except json.JSONDecodeError as e:
        raise ValueError(
            f"Invalid JSON format in {path}: {e}"
        )
```

Good Error Message

- ✓ Clear and specific
- ✓ Tells you what failed
- ✓ Includes the file, host or value involved
- ✓ Helps you fix the issue

Bad Error Message

- ✗ Just "Error"
- ✗ No context
- ✗ Not helpful
- ✗ Hard to debug

6. Cleanup and Fail-Safe Behavior

Use **finally** or context managers to ensure cleanup.

Using finally:

```
try:
    f = open("output.txt", "w")
    f.write("Processing data...")
except Exception as e:
    print(f"Error: {e}")
finally:
    f.close() # always closes
```

Using with (Recommended):

```
with open("output.txt", "w") as f:
    f.write("Processing data...")
# file is automatically closed
```

- ✓ Always close files and connections
- ✓ Clean up temporary files
- ✓ Leave the system in a safe state
- ✓ Use context managers if possible

7. Real Automation Example

Check if a service is running with proper error handling.

```
import subprocess

def check_service(service: str) -> bool:
    try:
        result = subprocess.run(["systemctl", "is-active", service],
                                capture_output=True, text=True, check=True)
        return result.stdout.strip() == "active"
    except subprocess.CalledProcessError:
        print(f"Service {service} is not running")
        return False
    except Exception as e:
        print(f"Unexpected error checking {service}: {e}")
        return False
```



Example Usage

```
if check_service("nginx"):
    print("Nginx is running")
else:
    print("Nginx is not running")
```

8. Key Takeaways

- ✓ Use try, except, else, finally to handle errors
- ✓ Catch specific exceptions when possible
- ✓ Raise exceptions for invalid input or unsafe conditions
- ✓ Provide clear and actionable error messages
- ✓ Always clean up resources (files, connections, temporary data)
- ✓ Handle both expected and unexpected failures
- ✓ Make your automation scripts fail-safe and reliable

9. Common Beginner Mistakes

- ✗ Using a bare except without specifying the exception
- ✗ Hiding errors instead of handling them
- ✗ Not providing helpful error messages
- ✗ Not cleaning up files or connections
- ✗ Assuming things always work
- ✗ Not handling unexpected exceptions
- ✗ Letting the script continue in an unsafe state



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

11

Running Linux Commands with Python

subprocess, arguments, stdout, stderr, return codes, timeouts, shell=False, command failures, and avoiding command-injection mistakes.

1. Running a Simple Command

Use `subprocess.run()` to execute a Linux command.

```
import subprocess
result = subprocess.run(
    ["uname", "-a"],
    capture_output=True,
    text=True,
    check=True
)
print("Output:", result.stdout) # Linux system info
```

Key Points

- ✓ Use a list of arguments
- ✓ `capture_output=True` captures stdout and stderr
- ✓ `text=True` returns strings
- ✓ `check=True` raises an error if the command fails

2. Accessing stdout, stderr and Return Code

Get command output, error output, and exit status.

```
import subprocess
result = subprocess.run(
    ["ls", "/var/log"],
    capture_output=True,
    text=True
)
print("Return code:", result.returncode)
print("STDOUT:", result.stdout)
print("STDERR:", result.stderr)
```

Return Codes

0	Success
1	General error
2	Misuse of shell command
126	Permission denied
127	Command not found



3. Handling Command Failures

Use `check=True` or handle errors manually.

```
import subprocess
try:
    result = subprocess.run(
        ["systemctl", "status", "nginx"],
        capture_output=True, text=True, check=True
    )
    print(result.stdout)
except subprocess.CalledProcessError as e:
    print("Command failed with exit code:", e.returncode)
    print("Error:", e.stderr)
```

Common Failures

- ✗ Command not found
- ✗ Permission denied
- ✗ Non-zero exit code
- ✗ Service not running
- ✗ Invalid arguments

4. Using Timeouts

Set a timeout to avoid hanging commands.

```
import subprocess
try:
    result = subprocess.run(
        ["ping", "-c", "4", "google.com"],
        capture_output=True,
        text=True,
        timeout=10
    )
    print(result.stdout)
except subprocess.TimeoutExpired:
    print("Command timed out!")
```

Timeout Tips

- ✓ Set an appropriate timeout value
- ✓ Handle `TimeoutExpired`
- ✓ Use timeouts for external commands that may hang



5. Using shell=False (Recommended)

Always use `shell=False` and pass arguments as a list.

Correct (Safe)

```
subprocess.run(
    ["ls", "-l", "/var/log"],
    shell=False
)
```

- ✓ Prevents command injection
- ✓ Handles arguments correctly
- ✓ Secure and recommended

Avoid (Unsafe)

```
subprocess.run(
    "ls -l /var/log",
    shell=True
)
```

- ✗ Can be vulnerable to command injection
- ✗ Use only when necessary
- ✗ Harder to handle arguments

6. Avoiding Command Injection

Never build shell commands using user input.

Vulnerable (Do NOT do this)

```
import subprocess
user_input = "rm -rf /" # malicious input
subprocess.run(
    f"ls {user_input}",
    shell=True
)
```

Command Injection Risks

- ✗ Attackers can run multiple commands
- ✗ Can delete files or access sensitive data
- ✗ Never use user input with `shell=True`

Safe (Use this instead)

```
import subprocess
user_input = "-l"
subprocess.run(
    ["ls", user_input],
    shell=False
)
```

How to Stay Safe

- ✓ Use `shell=False`
- ✓ Pass a list of arguments
- ✓ Validate or sanitize user input
- ✓ Use allowlists for permitted values

7. Practical Automation Examples

Run real Linux commands for system administration tasks.

```
import subprocess
commands = [
    ["df", "-h"],
    ["free", "-m"],
    ["systemctl", "is-active", "nginx"],
    ["whoami"]
]
for cmd in commands:
    result = subprocess.run(cmd, capture_output=True, text=True)
    print(f"$ {' '.join(cmd)}")
    print("Return code:", result.returncode)
    print("Output:\n", result.stdout)
```

Example Output:

```
$ df -h
Return code: 0
Filesystem Size Used Avail ...

$ systemctl is-active nginx
Return code: 0
Output:
active
```

8. Key Takeaways

- ✓ Use `subprocess.run()` to execute Linux commands
- ✓ Pass arguments as a list and use `shell=False`
- ✓ Capture and check stdout, stderr and return codes
- ✓ Handle failures with `try/except` or `check=True`
- ✓ Use timeouts to avoid hanging commands
- ✓ Never use unsanitized user input
- ✓ Always validate inputs and use allowlists
- ✓ Use clear and helpful error messages
- ✓ These practices make your automation scripts safe, reliable and production ready

9. Common Beginner Mistakes

- ✗ Using `shell=True` with user input
- ✗ Not checking the return code
- ✗ Ignoring stderr (error messages)
- ✗ Not using timeouts
- ✗ Not handling command failures
- ✗ Building commands with string concatenation
- ✗ Assuming a command always succeeds
- ✗ Not validating or sanitizing user input
- ✗ Using `subprocess` without understanding arguments



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

12

Environment Variables, Secrets & Configuration

os.environ, configuration separation, .env concepts, credentials, secret exposure risks, defaults, validation, and never hardcoding passwords or API keys.

1. Reading Environment Variables

Use os.environ to read environment variables.

```
import os
# Get a value
db_host = os.environ.get("DB_HOST")
db_port = os.environ.get("DB_PORT",
"5432")
# Required variable (raise error if
missing)
api_key = os.environ["API_KEY"]
print(f"Host: {db_host}")
print(f"Port: {db_port}")
```

Key Points

- ✓ os.environ.get() returns None if missing
- ✓ You can set a default
- ✓ Use os.environ["KEY"] for required values
- ✓ Environment variables should not contain secrets in code

2. Using a .env File (Development)

Use a .env file to keep configuration outside your code.

```
from dotenv import load_dotenv
import os
# Load variables from .env file
load_dotenv()

db_user = os.getenv("DB_USER")
db_password = os.getenv("DB_PASSWORD")

print(f"User: {db_user}")
```

Example .env file

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=admin
DB_PASSWORD=secret123
API_KEY=your_api_key
ENV=dev
```



3. Configuration Separation

Use different configuration for different environments.

```
import os
env = os.getenv("ENV", "dev")
if env == "prod":
    db_host = os.getenv("DB_HOST")
else:
    db_host = "localhost"
print(f"Environment: {env}")
print(f"DB Host: {db_host}")
```

Best Practice

- ✓ Keep configuration outside the code
- ✓ Use different values for dev, staging and production
- ✓ Make your code environment agnostic

dev staging production

4. Avoid Hardcoding Secrets

Never hardcode passwords, API keys or secrets in your code.

✗ Wrong (Never do this)

```
# DO NOT DO THIS
api_key = "sk-123456789"
db_password = "mypassword"
token = "abcd1234"
```



✓ Correct (Use environment variables)

```
import os
api_key = os.getenv("API_KEY")
db_password =
os.getenv("DB_PASSWORD")
token = os.getenv("TOKEN")
```



5. Validation and Defaults

Validate required configuration and provide defaults.

```
import os
def get_required_env(name: str) -> str:
    value = os.getenv(name)
    if not value:
        raise ValueError(f"Missing required
environment variable: {name}")
    return value

db_host = get_required_env("DB_HOST")
db_port = int(os.getenv("DB_PORT", "5432"))
debug = os.getenv("DEBUG", "false").lower() == "true"
print(db_host, db_port, debug)
```

Validation Tips

- ✓ Check that required variables exist
- ✓ Provide sensible defaults
- ✓ Convert to correct data types
- ✓ Fail early with clear error messages



6. Secret Exposure Risks

Be aware of common ways secrets can be exposed.

- ✗ Hardcoding secrets in code
- ✗ Committing .env files to Git
- ✗ Printing secrets in logs
- ✗ Storing secrets in plain text files
- ✗ Sharing screenshots with secrets
- ✗ Using world-readable files
- ✗ Exposing secrets in error messages
- ✗ Using environment variables in untrusted systems



7. Practical Example: Using Configuration in a Script

A complete example that loads configuration and uses it safely.

```
from dotenv import load_dotenv
import os
import requests
load_dotenv()

api_url = os.getenv("API_URL", "https://api.example.com")
api_key = os.getenv("API_KEY")
if not api_key:
    raise ValueError("API_KEY is required")

headers = {"Authorization": f"Bearer {api_key}"}
response = requests.get(f"{api_url}/status", headers=headers, timeout=10)
print(f"Status: {response.status_code}")
```



8. Key Takeaways

- ✓ Use environment variables for configuration
- ✓ Keep secrets out of your code
- ✓ Use .env files for local development
- ✓ Validate required variables
- ✓ Provide sensible defaults
- ✓ Never hardcode passwords or API keys
- ✓ Be aware of secret exposure risks
- ✓ Use different configuration per environment
- ✓ Fail early with clear error messages
- ✓ Keep your automation scripts secure and portable

9. Common Beginner Mistakes

- ✗ Hardcoding secrets in the script
- ✗ Committing .env files to Git
- ✗ Not validating required variables
- ✗ Using wrong variable names
- ✗ Forgetting default values
- ✗ Printing sensitive information
- ✗ Using environment variables without type conversion
- ✗ Assuming variables always exist
- ✗ Not separating configuration for different environments



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

13

Logs, Timestamps & Automation Evidence

Python logging, log levels, timestamps, useful context, recording success and failure, log files, and why `print()` alone is insufficient for operational automation.

1. Why Logging Matters

Logging gives you a clear record of what happened.

- ✓ Tracks what your script did
- ✓ Helps with troubleshooting
- ✓ Records success and failure
- ✓ Includes timestamps and context
- ✓ Useful for monitoring and auditing
- ✓ Essential for production automation

Why `print()` is not enough

- ✗ No log levels
- ✗ No timestamp by default
- ✗ No consistent format
- ✗ Hard to filter or search
- ✗ Not written to a file automatically
- ✗ Difficult to use in production

2. Basic Logging Setup

Set up Python logging with timestamp, level and message.

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    filename='app.log',
    filemode='a'
)
logging.info("Script started")
logging.warning("This is a warning")
logging.error("This is an error")
```

Output (app.log)

```
2026-09-27 10:15:23 |
INFO | Script started
2026-09-27 10:15:25 |
WARNING | This is a
warning
2026-09-27 10:15:27 |
ERROR | This is an error
```



3. Log Levels

Different log levels for different types of messages.

Level	When to Use	Example
DEBUG	Detailed information for troubleshooting	<code>logging.debug("Variable value: %s", value)</code>
INFO	Normal operational messages	<code>logging.info("Task completed")</code>
WARNING	Something unexpected but script can continue	<code>logging.warning("Disk space low")</code>
ERROR	An error occurred, operation failed	<code>logging.error("Failed to connect to DB")</code>
CRITICAL	A serious error, script cannot continue	<code>logging.critical("System is unavailable")</code>

4. Adding Useful Context

Include useful information like host, script name and function.

```
import logging
import socket
import os

hostname = socket.gethostname()
script_name = os.path.basename(__file__)

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    filename='automation.log'
)

logging.info(f"[{hostname}] [{script_name}] Starting task")
logging.info(f"[{hostname}] Checking disk space")
logging.info(f"[{hostname}] Task completed successfully")
```

Benefits

- ✓ Shows which server ran the script
- ✓ Shows which script produced the log
- ✓ Makes it easy to find issues later
- ✓ Useful when running scripts on multiple servers



5. Logging to Console and File

Write logs to both the console and a file.

```
import logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

# Console handler
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)

# File handler
file_handler = logging.FileHandler('app.log')
file_handler.setLevel(logging.INFO)

# Log format
formatter = logging.Formatter('%(asctime)s | %(levelname)s | %(message)s')
console_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

# Add handlers
logger.addHandler(console_handler)
logger.addHandler(file_handler)

logging.info("Logging to both console and file")
```

Output

```
2026-09-27 10:20:11 |
INFO | Logging to both
console and file
```



6. Logging Success and Failure

Example: running a command and logging the result.

```
import logging
import subprocess

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s',
    filename='command.log'
)

try:
    result = subprocess.run(['ls', '-lh', '/var/log'],
        capture_output=True, text=True, check=True)
    logging.info("Command succeeded")
    logging.info("Output:\n%s", result.stdout)
except subprocess.CalledProcessError as e:
    logging.error("Command failed")
    logging.error("Return code: %s", e.returncode)
    logging.error("Error output: %s", e.stderr)
```

What Gets Logged?

- ✓ When the command started
- ✓ If it succeeded or failed
- ✓ Return code on failure
- ✓ Standard output (stdout)
- ✓ Error output (stderr)
- ✓ Timestamp and context



7. Using Timestamps Manually

You can also add timestamps manually if needed.

```
from datetime import datetime
now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
print(f"[now] | Custom log message")
```

Example Output

```
2026-09-27 10:30:45 |
Custom log message
```

8. Key Takeaways

- ✓ Use the logging module, not just `print()`
- ✓ Include timestamps, log levels and useful context
- ✓ Write logs to a file for historical records
- ✓ Log both success and failure with enough details
- ✓ Use different log levels appropriately
- ✓ Make logs easy to read and search
- ✓ Logging is essential for operational automation

9. Common Beginner Mistakes

- ✗ Using only `print()` for logging
- ✗ Not including timestamps
- ✗ Logging sensitive information (passwords)
- ✗ Not writing logs to a file
- ✗ Using the wrong log level
- ✗ Logging too little or too much information
- ✗ Not handling and logging errors
- ✗ Making logs hard to read or search



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

HTTP APIs & Automation with Python

Requests, endpoints, HTTP methods, headers, authentication concepts, status codes, JSON responses, timeouts, pagination, retries, and basic API error handling.

1. What is an HTTP API?

An API (Application Programming Interface) allows you to access data or perform actions over HTTP.

Python Script

- Send request
- Get response
- Process data

HTTP Request (GET, POST, etc.)



API Server

- Endpoint
- Data
- Actions

3. Making a Basic GET Request

Use the requests library to send a GET request.

```
import requests

url = "https://api.github.com/users/veriqta"
response = requests.get(url, timeout=10)

print("Status Code:", response.status_code)
print("Response JSON:")
print(response.json())
```

Key Points

- ✓ Use `requests.get()`
- ✓ Check status code
- ✓ Use `timeout`
- ✓ Parse JSON with `response.json()`

2. HTTP Methods (Common Ones)

Different methods are used for different actions.

Method	Purpose	Example	Method Examples
GET	Retrieve data	<code>requests.get(url)</code>	GET /users
POST	Create new data	<code>requests.post(url, json=data)</code>	POST /users
PUT	Update existing data	<code>requests.put(url, json=data)</code>	PUT /users/1
PATCH	Partially update data	<code>requests.patch(url, json=data)</code>	PATCH /users/1
DELETE	Delete data	<code>requests.delete(url)</code>	DELETE /users/1

4. Adding Headers and Authentication

APIs often require headers and authentication.

```
import requests

url = "https://api.example.com/data"
headers = {
    "Authorization": "Bearer YOUR_TOKEN",
    "Accept": "application/json"
}
response = requests.get(url, headers=headers,
                        timeout=10)
print(response.json())
```

Auth Types (Common)

- 1 API Key
- 2 Bearer Token (used in headers)
- 3 Basic Authentication
- 4 OAuth (used for secure access)



5. Working with JSON Responses

Most APIs return data in JSON format.

```
import requests

url = "https://api.github.com/repos/veriqta/DevOps-World"
response = requests.get(url, timeout=10)
data = response.json() # Convert JSON to Python dict

print("Repository:", data["name"])
print("Stars:", data["stargazers_count"])
print("Description:", data["description"])
```

Example JSON Response

```
{
  "name": "DevOps-World",
  "stargazers_count": 3500,
  "description": "DevOps resources",
  "owner": {
    "login": "veriqta"
  }
}
```

6. Handling Status Codes

Check the response status code and handle different cases.

```
import requests

url = "https://api.example.com/data"
response = requests.get(url, timeout=10)
if response.status_code == 200:
    print("Success:", response.json())
elif response.status_code == 404:
    print("Resource not found")
elif response.status_code == 401:
    print("Unauthorized. Check your token.")
else:
    print(f"Error: {response.status_code}")
```

Common Status Codes

- | | |
|-----|-------------------|
| 200 | OK (Success) |
| 201 | Created |
| 400 | Bad Request |
| 401 | Unauthorized |
| 403 | Forbidden |
| 404 | Not Found |
| 429 | Too Many Requests |
| 500 | Server Error |

7. Timeouts, Retries and Error Handling

Use timeouts, retries and proper error handling for reliability.

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
retries = Retry(total=3, backoff_factor=1,
               status_forcelist=[429, 500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

try:
    response = session.get("https://api.example.com/data",
                          timeout=10)
    response.raise_for_status() # Raises an error for 4xx/5xx
    data = response.json()
    print("Success:", data)
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.RequestException as e:
    print("Request failed: {e}")
```

Retry Benefits

- ✓ Handles temporary failures
- ✓ Useful for rate limits (429)
- ✓ Improves reliability
- ✓ Uses exponential backoff



8. Pagination (Getting All Results)

Some APIs return data in pages.

```
import requests

url = "https://api.example.com/items"
params = {"page": 1, "per_page": 100}
all_items = []

while True:
    response = requests.get(url, params=params,
                          timeout=10)
    data = response.json()
    items = data["items"]
    all_items.extend(items)
    if not data.get("next_page"):
        break
    params["page"] = data["next_page"]

print(f"Total items: {len(all_items)}")
```

Pagination Tips

- ✓ Check API documentation
- ✓ Look for `next_page`, `page`, `links`, or `cursor`
- ✓ Keep requesting until no more pages
- ✓ Handle rate limits
- ✓ Combine all results



9. Key Takeaways

- ✓ Use the requests library for HTTP APIs
- ✓ Understand HTTP methods and status codes
- ✓ Always set a `timeout`
- ✓ Use headers and proper authentication
- ✓ Parse JSON responses and handle errors
- ✓ Implement retries for reliability
- ✓ Handle pagination for large datasets
- ✓ Check status codes and validate responses
- ✓ Never expose API keys or tokens in code
- ✓ Build automation that can handle failures

10. Common Beginner Mistakes

- ✗ Not setting a `timeout`
- ✗ Not checking the status code
- ✗ Assuming the response is always JSON
- ✗ Not handling errors or retries
- ✗ Hardcoding API keys or tokens
- ✗ Ignoring pagination
- ✗ Using the wrong HTTP method
- ✗ Not reading API documentation
- ✗ Exposing secrets in code
- ✗ Not validating the response data



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



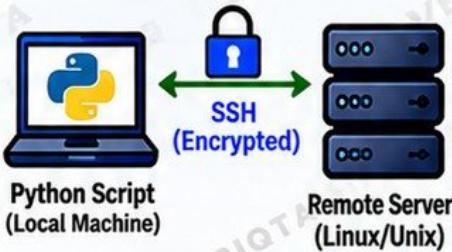
@veriqta

Automating Servers & Remote Operations

SSH concepts, remote execution, inventories, safe command execution, connection failures, authentication failures, and when tools such as Ansible are a better choice.

1. Understanding SSH and Remote Operations

SSH (Secure Shell) allows you to securely connect to and run commands on remote servers.



Key Points

- ✓ Secure connection
- ✓ Authenticate user
- ✓ Execute commands
- ✓ Manage multiple servers
- ✓ Get output and errors

2. Connecting to a Remote Server (Using paramiko)

Use the paramiko library for SSH connections.

```
import paramiko
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(
    paramiko.AutoAddPolicy())
ssh.connect(hostname="192.168.1.10",
            username="ubuntu",
            key_filename=~/.ssh/id_rsa")
stdin, stdout, stderr = ssh.exec_command(
    "hostname")
print("Output:", stdout.read().decode())
print("Error:", stderr.read().decode())
ssh.close()
```

Key Points

- ✓ Use key-based authentication or password (less secure)
- ✓ Handle connection errors
- ✓ Close the connection after use



3. Running Commands Remotely

Execute commands and get the output.

```
command = "df -h"
stdin, stdout, stderr = ssh.exec_command(command)
exit_code = stdout.channel.recv_exit_status()
output = stdout.read().decode()
error = stderr.read().decode()
print("Exit Code:", exit_code)
print("Output:\n", output)
print("Error:\n", error)
```

Check Exit Code

- ✓ 0 = success
- ✓ Non-zero = command failed
- ✓ Always check both stdout and stderr
- ✓ Handle failures properly

4. Using an Inventory (Multiple Servers)

Manage multiple servers using a simple inventory list (JSON/YAML).

```
# inventory.json
{
  "servers": [
    { "name": "web1", "host": "192.168.1.10" },
    { "name": "web2", "host": "192.168.1.11" },
    { "name": "db1", "host": "192.168.1.12" }
  ]
}
import json
with open("inventory.json") as f:
    data = json.load(f)
for server in data["servers"]:
    print(f"Connecting to {server['name']}")
    ({server['host']})
# connect and run commands here
```

Benefits

- ✓ Easily manage many servers
- ✓ Reuse the same script for multiple hosts
- ✓ Store hosts, users and other details



5. Safe Command Execution

Avoid command injection and unsafe practices.

Good (Safe)

```
command = ["ls", "-l", "/var/log"]
ssh.exec_command(" ".join(command))
```

- ✓ Use a list or validate input
- ✓ Run only necessary commands
- ✓ Avoid passing user input directly

Bad (Unsafe)

```
user_input = input("Enter file: ")
ssh.exec_command(f"ls {user_input}")
```

- ✗ Can allow command injection
- ✗ Do not trust user input
- ✗ Validate and sanitize inputs

6. Common Failures and Troubleshooting

Handle and troubleshoot common connection and execution issues.

Issue	Cause	How to Fix
Connection refused	Server not reachable	Check host, network, firewall, and SSH service
Authentication failed	Wrong key or password	Verify username, key file, permissions
Timeout	Network issues or slow host	Increase timeout, check network
Permission denied	User has no sudo rights	Use correct user or sudo
Command not found	Binary not in PATH	Use full path to the command
Host key verification failed	Unknown host key	Accept or add the host key (or use AutoAddPolicy)
Non-zero exit code	Command failed	Check stderr, handle errors

7. When to Use Ansible Instead

For complex automation, Ansible is often a better choice.

Use Ansible when:

- ✓ Managing many servers (large inventory)
- ✓ Need configuration management
- ✓ Want idempotent (safe) operations
- ✓ Need file transfers, package management, and more
- ✓ Prefer YAML playbooks instead of custom code



ANSIBLE

Benefits

- ✓ Simpler to manage
- ✓ Scales well
- ✓ Many built-in modules
- ✓ Widely used in DevOps and production

8. Key Takeaways

- ✓ Use SSH for secure remote operations
- ✓ Always handle connection and authentication errors
- ✓ Check command exit codes
- ✓ Use inventories for multiple servers
- ✓ Validate and sanitize any user input
- ✓ Use timeouts to avoid hanging scripts
- ✓ Consider Ansible for larger and more complex automation tasks

9. Common Beginner Mistakes

- ✗ Hardcoding passwords or keys
- ✗ Not handling connection failures
- ✗ Not checking exit codes
- ✗ Ignoring stderr output
- ✗ Using unsafe user input
- ✗ Not closing SSH connections
- ✗ Assuming commands always exist
- ✗ Not using timeouts
- ✗ Reinventing complex automation instead of using Ansible



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DevOps Automation with Git, CI/CD & Cloud

Automating repository tasks, interacting with CI/CD systems, cloud SDK concepts, AWS boto3 examples, idempotency, credentials, and API-driven infrastructure operations.

1. Automating Git Repository Tasks

Use GitPython to automate common repository operations.

```
from git import Repo

# Clone a repository
repo = Repo.clone_from(
    "https://github.com/veriqta/demo.git",
    "demo"
)

# Make a change and commit
file_path = "demo/README.md"
with open(file_path, "a") as f:
    f.write("\nUpdated by automation\n")
repo.index.add([file_path])
repo.index.commit("Automated update")
repo.remote().push()
```

Key Points

- ✓ Clone, commit, push automatically
- ✓ Use tokens or SSH keys for authentication
- ✓ Automate release notes, tags and branches
- ✓ Handle errors and conflicts
- ✓ Never hardcode credentials



2. Interacting with CI/CD Systems

Trigger and monitor CI/CD pipelines using APIs.

```
import requests

url = "https://gitlab.example.com/api/v4/projects/1/pipelines"
headers = {"PRIVATE-TOKEN": "your_token"}
data = {"ref": "main"}

response = requests.post(url, headers=headers,
                        json=data, timeout=10)

print("Status Code:", response.status_code)
print("Pipeline ID:", response.json().get("id"))
```

Popular CI/CD APIs



3. Working with Cloud SDKs (AWS boto3)

Use boto3 to automate AWS infrastructure and services.

```
import boto3

# Create an EC2 instance (example)
ec2 = boto3.client("ec2", region_name="us-east-1")
response = ec2.run_instances(
    ImageId="ami-0abcd1234efgh5678",
    InstanceType="t2.micro",
    MinCount=1,
    MaxCount=1,
    TagSpecifications=[{
        'ResourceType': 'instance',
        'Tags': [{ 'Key': 'Name', 'Value': 'DevOps-Automation' }]
    }]
)
print("Instance ID:", response["Instances"][0]["InstanceId"])
```

AWS Services (Examples)

- ✓ EC2 (compute)
- ✓ S3 (storage)
- ✓ IAM (access control)
- ✓ RDS (databases)
- ✓ VPC (networking)
- ✓ CloudWatch (monitoring)
- ✓ And many more...



4. Idempotency in Automation

Ensure your automation can run multiple times safely.

```
import boto3
ec2 = boto3.client("ec2")

# Check if a security group already exists
response = ec2.describe_security_groups(
    Filters=[{"Name": "group-name",
              "Values": ["web-sg"]}])

if response["SecurityGroups"]:
    print("Security group already exists.")
else:
    ec2.create_security_group(
        GroupName="web-sg",
        Description="Web security group"
    )
    print("Security group created.")
```

Why Idempotency?

- ✓ Safe to run multiple times
- ✓ Prevents duplicate resources
- ✓ More reliable and easier to maintain
- ✓ Important for CI/CD and infrastructure automation



5. Managing Credentials Securely

Use environment variables, IAM roles or secret managers.

```
import os
import boto3

# Use environment variables (better than hardcoding)
aws_access_key = os.getenv("AWS_ACCESS_KEY_ID")
aws_secret_key = os.getenv("AWS_SECRET_ACCESS_KEY")

session = boto3.Session(
    aws_access_key_id=aws_access_key,
    aws_secret_access_key=aws_secret_key,
    region_name="us-east-1"
)
s3 = session.client("s3")
```

Never Do This

```
aws_access_key = "AKIA..."
aws_secret_key = "abcd1234..."
```

Use Instead

- ✓ Environment variables
- ✓ IAM roles (on AWS)
- ✓ Secrets Manager
- ✓ Configuration files (.env)



6. API-Driven Infrastructure Operations

Create, update and delete cloud resources using APIs.

```
import boto3
ec2 = boto3.client("ec2")

# Example: Stop an instance
instance_id = "i-0123456789abcdef0"
response = ec2.stop_instances(
    InstanceIds=[instance_id]
)

print("Stopping instance:", instance_id)
print("Current state:", response["StoppingInstances"][0]
      ["CurrentState"]["Name"])
```

Common Operations

- ✓ Create resources
- ✓ Update existing resources
- ✓ Delete resources
- ✓ Query and list resources
- ✓ Handle API errors
- ✓ Implement retries
- ✓ Use tags for management
- ✓ Combine with CI/CD pipelines



7. Putting It All Together (Example Workflow)

Automate a complete DevOps workflow.



8. Key Takeaways

- ✓ Automate Git repository tasks
- ✓ Use CI/CD APIs to trigger and monitor pipelines
- ✓ Use cloud SDKs like boto3 for infrastructure automation
- ✓ Implement idempotency for safe automation
- ✓ Manage credentials securely
- ✓ Handle errors and retries
- ✓ Combine tools for end-to-end DevOps workflows
- ✓ Always follow security best practices

9. Common Beginner Mistakes

- ✗ Hardcoding credentials
- ✗ Not handling API errors
- ✗ Not using timeouts
- ✗ Creating duplicate resources
- ✗ Ignoring idempotency
- ✗ Not validating responses
- ✗ Using overly permissive permissions
- ✗ Not understanding costs
- ✗ Skipping logging and monitoring
- ✗ Running automation without testing



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Scheduling & Production-Ready Scripts

cron, scheduled jobs, exit codes, retries, timeouts, locking, duplicate execution, idempotency, permissions, observability, and rollback thinking.

1. What is Task Scheduling?

Task scheduling runs your scripts automatically at a specific time or interval.



Common Use Cases

- ✓ Backups
- ✓ Health checks
- ✓ Log rotation
- ✓ Data processing
- ✓ API synchronization
- ✓ Infrastructure tasks
- ✓ Reports and alerts

2. Using cron for Scheduled Jobs

cron is a time-based scheduler in Linux.

Edit your crontab

`crontab -e`

Common cron format

m h dom mon dow command
0 2 * * * /usr/bin/python3 /scripts/backup.py

Examples

0 2 * * * Run daily at 2 AM
0 */6 * * * Run every 6 hours
30 1 * * 1 Run every Monday at 1:30 AM

Best Practices

- ✓ Use full paths
- ✓ Redirect output to log files
- ✓ Use a virtual environment
- ✓ Set proper permissions
- ✓ Test your script manually first

3. Handling Exit Codes

Exit codes tell you if a script succeeded or failed.

```

import sys
def main():
    try:
        # your script logic
        print("Task completed successfully")
        return 0 # success
    except Exception as e:
        print(f"Error: {e}")
        return 1 # failure

if __name__ == "__main__":
    sys.exit(main())
  
```

Common Exit Codes

- 0 Success
- 1 General error
- 2 Invalid arguments
- 3 Partial failure
- 4 Dependency error
- 5 External service error

4. Timeouts, Retries and Error Handling

Use timeouts and retries to make your scripts more reliable.

```

import requests
import time
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
retries = Retry(
    total=3,
    backoff_factor=2,
    status_forcelist=[429, 500, 502, 503, 504]
)
session.mount("https://", HTTPAdapter(max_retries=retries))

response = session.get("https://api.example.com", timeout=10)
print(response.status_code)
  
```

Key Points

- ✓ Set a reasonable timeout
- ✓ Use retries with backoff
- ✓ Handle specific
- ✓ Log failures

5. Preventing Duplicate Execution (Locking)

Use file locking to prevent multiple instances from running.

```

import fcntl
import time
import sys

lock_file = "/tmp/my_script.lock"
with open(lock_file, "w") as f:
    try:
        fcntl.flock(f, fcntl.LOCK_EX | fcntl.LOCK_NB)
        print("Script is running...")
        time.sleep(60) # your script logic
    except BlockingIOError:
        print("Another instance is already running")
        sys.exit(1)
  
```

Benefits

- ✓ Prevents duplicate runs
- ✓ Avoids data corruption
- ✓ Useful for long-running tasks
- ✓ Simple and effective



6. Idempotency in Automation

Idempotent scripts can run multiple times safely without unwanted changes.

```

import boto3
ec2 = boto3.client("ec2")
instance_id = "i-0123456789abcdef0"
# Check if instance is already running
response = ec2.describe_instances(
    InstanceIds=[instance_id]
)
state = response["Reservations"][0]["Instances"][0][
    "State"]["Name"]
if state != "running":
    ec2.start_instances(InstanceIds=[instance_id])
    print("Instance started")
else:
    print("Instance already running (idempotent)")
  
```

Key Points

- ✓ Check current state before making changes
- ✓ Safe to run multiple times
- ✓ Avoid creating duplicate resources
- ✓ Essential for reliable automation



7. Permissions and Security

Use the correct permissions and least privilege access.

- ✓ Use dedicated user accounts
- ✓ Set correct file permissions (chmod)
- ✓ Protect sensitive configuration files
- ✓ Use environment variables or secret managers
- ✓ Limit cloud permissions using IAM roles/policies
- ✓ Avoid running scripts as root unless necessary

Set file permissions

`chmod +x /scripts/my_script.py`
`chmod 700 /scripts/my_script.py`



8. Observability and Logging

Record important information for monitoring and debugging.

- ✓ Use Python logging module
- ✓ Include timestamps and log levels
- ✓ Log both success and failure
- ✓ Write logs to files (not just print)
- ✓ Include useful context (host, script, task)
- ✓ Consider log rotation



9. Rollback Thinking

Plan for what to do when something fails.

- ✗ Understand the impact of changes
- ✗ Keep backups where possible
- ✗ Use safe, incremental changes
- ✗ Have a rollback or recovery plan
- ✗ Test scripts in a non-production environment
- ✗ Log actions so you can reverse if needed



10. Key Takeaways

- ✓ Use cron to schedule your scripts
- ✓ Always handle exit codes
- ✓ Use timeouts and retries
- ✓ Prevent duplicate execution with locking
- ✓ Make scripts idempotent
- ✓ Use proper permissions and secure credentials
- ✓ Add logging and monitoring
- ✓ Think about rollback and recovery
- ✓ Test before running in production

Common Beginner Mistakes

- ✗ Not handling failures or exit codes
- ✗ Running multiple instances at the same time
- ✗ No logging or observability
- ✗ Using hardcoded credentials
- ✗ Ignoring timeouts and retries
- ✗ Not making scripts idempotent
- ✗ Incorrect file permissions
- ✗ No rollback plan
- ✗ Testing directly in production



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

18

Troubleshooting Python Automation

Syntax errors, import failures, missing packages, permission errors, path problems, API failures, timeouts, malformed data, debugging with evidence, and verifying fixes.

1. Syntax Errors

Syntax errors happen when Python code is not written correctly.

Example (Syntax Error)

```
if True
print("Missing colon")
```

File "script.py", line 1

```
if True
^
```

SyntaxError: expected ':'

How to Fix

- ✓ Check line numbers
- ✓ Fix missing colons, brackets, or quotes
- ✓ Use a code editor with linting
- ✓ Run the script again



2. Import Failures and Missing Packages

Occurs when a module is not installed or cannot be found.

Example

```
import requests
```

ModuleNotFoundError:
No module named 'requests'

Fix

- ✓ Install the package
`pip install requests`
- ✓ Check the correct environment
`python3 -m pip install requests`
- ✓ Verify installation
`pip show requests`



3. Permission Errors

Happens when the script does not have the required permissions.

Example

```
with open("/var/log/app.log", "w") as f:
    f.write("test")
```

PermissionError: [Errno 13]
Permission denied: '/var/log/app.log'

Fix

- ✓ Use a directory you have access to
- ✓ Run with correct permissions (chmod)
- ✓ Use sudo only when necessary
- ✓ Check file and directory ownership



4. Path Problems

Incorrect or relative paths can cause file not found errors.

Example

```
with open("data/input.csv") as f:
    print(f.read())
```

FileNotFoundError: [Errno 2]
No such file or directory: 'data/input.csv'

Fix

- ✓ Use absolute paths or correct relative paths
- ✓ Check the current working directory
- ✓ Verify the file exists
- ✓ Use `os.path` to handle paths safely



5. API Failures

APIs can return errors due to invalid requests, authentication problems or server issues.

Example

```
import requests
url = "https://api.example.com/data"
response = requests.get(url)
print(response.json())
```

requests.exceptions.HTTPError:
403 Client Error: Forbidden
for url: https://api.example.com/data

Fix

- ✓ Check status code and error message
- ✓ Verify authentication (API key, token)
- ✓ Validate request parameters
- ✓ Read API documentation



6. Timeouts

A request or operation takes too long and fails.

Example

```
import requests
response = requests.get("https://api.example.com",
    timeout=5)
```

requests.exceptions.Timeout:
HTTPConnectionPool(host='api.example.com',
port=443): Read timed out.

Fix

- ✓ Increase timeout value
- ✓ Use retries with backoff
- ✓ Check network connectivity
- ✓ Verify the API or server is not slow or down



7. Malformed Data

Data may not be in the expected format (JSON, CSV, etc.).

Example

```
import json
data = {'name': 'veriqta'} # Invalid JSON (single quotes)
json.loads(data)
```

json.decoder.JSONDecodeError:
Expecting property name enclosed in double quotes

Fix

- ✓ Validate data format before processing
- ✓ Use correct JSON syntax (double quotes)
- ✓ Handle parsing errors with try/except
- ✓ Log the actual data for debugging



8. Debugging with Evidence

Collect logs, errors and useful information to find the real root cause.

Example

```
import logging
logging.basicConfig(level=logging.INFO)
try:
    # your code here
    pass
except Exception as e:
    logging.exception("Error occurred")
```

What to Collect

- ✓ Full error message and stack trace
- ✓ Input values and environment details
- ✓ Logs with timestamps
- ✓ API request and response (if safe)
- ✓ System information (Python version, OS)



9. Verifying Fixes

After fixing an issue, always test and confirm that it is resolved.

Steps to Verify

- 1 Re-run the script
- 2 Check expected output
- 3 Test with different inputs
- 4 Verify logs show success
- 5 Ensure no new errors appear
- 6 Monitor in a real environment (if applicable)



10. Common Errors and Solutions

Error	Cause	Solution
SyntaxError	Invalid Python syntax	Fix code syntax
ModuleNotFoundError	Missing package	Install the package
ImportError	Cannot import module	Check package and Python path
FileNotFoundError	File or path does not exist	Use correct path
PermissionError	Insufficient permissions	Check permissions or use sudo
HTTPError	API returned an error code	Check status code and request
Timeout	Operation took too long	Increase timeout, use retries
JSONDecodeError	Invalid JSON data	Validate and fix data format

11. Key Takeaways

- ✓ Read and understand the full error message
- ✓ Use logging to get evidence
- ✓ Check your environment (packages, paths, permissions)
- ✓ Validate inputs and external data (APIs, files)
- ✓ Handle errors gracefully with try/except
- ✓ Use timeouts and retries for reliability
- ✓ Verify fixes with proper testing
- ✓ Keep your scripts simple, clear and well logged

12. Common Beginner Mistakes

- ✗ Ignoring error messages
- ✗ Not installing required packages
- ✗ Using incorrect file paths
- ✗ Running scripts without proper permissions
- ✗ Not handling API errors
- ✗ Not setting timeouts
- ✗ Assuming data is always in the right format
- ✗ Not logging enough information
- ✗ Not verifying fixes
- ✗ Copying code without understanding it



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Build a Real DevOps Automation Tool

Capstone project combining configuration, filesystem operations, subprocess execution, API interaction, exception handling, logging, reporting, environment variables, validation, and exit codes. Finish with a production-readiness checklist.

1. Project Overview

Build a Python CLI tool that performs common DevOps tasks.

Tool Features

- ✓ Read configuration from environment variables and a config file
- ✓ Run system commands (subprocess)
- ✓ Interact with a cloud API (AWS example)
- ✓ Work with files and directories
- ✓ Handle errors and timeouts
- ✓ Log all activities with timestamps
- ✓ Generate a summary report
- ✓ Use proper exit codes
- ✓ Be production-ready

Example Use Cases

- ✓ Check disk space on servers
- ✓ Create a backup
- ✓ Start/stop a cloud instance
- ✓ Deploy an application
- ✓ Send a status report
- ✓ All in one automation tool



2. Project Structure

Organize your project properly.

devops_tool/

```
main.py
config.py
logger.py
commands.py
aws_ops.py
reporter.py
utils.py
requirements.txt
.env.example
README.md
```



3. Configuration and Environment Variables

Use environment variables and a config file.

config.py (example)

```
import os
from dotenv import load_dotenv

load_dotenv() # Load .env file

AWS_REGION = os.getenv("AWS_REGION",
                        "us-east-1")
BACKUP_DIR = os.getenv("BACKUP_DIR",
                        "/tmp/backups")
LOG_FILE = os.getenv("LOG_FILE",
                     "logs/devops_tool.log")
TIMEOUT = int(os.getenv("TIMEOUT", "30"))
```

.env.example

```
AWS_REGION=us-east-1
BACKUP_DIR=/tmp/backups
LOG_FILE=logs/devops_tool.log
TIMEOUT=30
AWS_ACCESS_KEY_ID=your_key
AWS_SECRET_ACCESS_KEY=your_secret
```



4. Logging Setup

Log all activities with timestamps and levels.

logger.py (example)

```
import logging
import os

def setup_logger():
    log_file = os.getenv("LOG_FILE")
    os.makedirs(os.path.dirname(log_file),
                exist_ok=True)
    logging.basicConfig(
        filename=log_file,
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s]
%(message)s"
    )
    return logging.getLogger("devops_tool")
```

Log Levels

- ✓ DEBUG - detailed info
- ✓ INFO - normal operations
- ✓ WARNING - potential issue
- ✓ ERROR - an error occurred
- ✓ CRITICAL - serious failure



5. Running System Commands

Use subprocess to execute Linux commands.

commands.py (example)

```
import subprocess

def run_command(command, timeout=30):
    try:
        result = subprocess.run(
            command,
            shell=False,
            capture_output=True,
            text=True,
            timeout=timeout
        )
        return result.returncode,
        result.stdout, result.stderr
    except subprocess.TimeoutExpired:
        return 124, "", "Command timed out"
```

6. Interacting with AWS (boto3)

Use boto3 to perform cloud operations.

aws_ops.py (example)

```
import boto3
from botocore.exceptions import
ClientError

def start_instance(instance_id):
    ec2 = boto3.client("ec2")
    try:
        response = ec2.start_instances(
            InstanceIds=[instance_id]
        )
        return True, response
    except ClientError as e:
        return False, str(e)
```



7. Main Script (Putting Everything Together)

Example workflow in main.py.

```
from logger import setup_logger
from config import BACKUP_DIR
from commands import run_command
from aws_ops import start_instance
from reporter import create_report

log = setup_logger()
log.info("DevOps tool started")

# Run a system command
code, out, err = run_command(["df -h"])
if code != 0:
    log.error(f"Command failed: {err}")
    exit(1)
else:
    log.info("Disk space check completed")

# Example: start an AWS instance
success, response = start_instance("i-0123456789abcdef0")
if not success:
    log.error(f"Failed to start instance: {response}")
    exit(2)
log.info("Instance started successfully")

create_report()
log.info("DevOps tool finished")
exit(0)
```

Exit Codes

- 0 Success
- 1 Command failure
- 2 API or cloud failure
- 3 Validation error
- 4 Configuration error

Benefits

- ✓ Handles real tasks
- ✓ Safe error handling
- ✓ Detailed logging
- ✓ Clear exit codes
- ✓ Easy to extend



8. Generating a Report

Create a summary report of what the tool did.

reporter.py (example)

```
import json
from datetime import datetime

def create_report(data=None):
    report = {
        "timestamp": datetime.now().isoformat(),
        "status": "completed",
        "details": data or {}
    }
    with open("report.json", "w") as f:
        json.dump(report, f, indent=4)
    return "report.json"
```

Sample Report Output

```
{
  "timestamp": "2026-09-27T10:30:00",
  "status": "completed",
  "details": {
    "disk_check": "success",
    "instance_started": "i-0123456789",
    "backup": "success"
  }
}
```



9. Validation and Error Handling

Validate inputs and handle exceptions.

utils.py (example)

```
import os

def validate_instance_id(instance_id):
    if not instance_id.startswith("i-"):
        raise ValueError("Invalid
instance ID")
    return True
```

Key Points

- ✓ Validate all user inputs
- ✓ Use try/except blocks
- ✓ Provide clear error messages
- ✓ Log the error details
- ✓ Exit with the correct code



10. Production-Readiness Checklist

Ensure your tool is ready for real use.

- ✓ Use environment variables (no hardcoded secrets)
- ✓ Validate all inputs and configurations
- ✓ Handle errors and exceptions properly
- ✓ Use timeouts and retries where needed
- ✓ Write detailed logs with timestamps
- ✓ Prevent duplicate executions if required
- ✓ Use idempotent operations
- ✓ Set correct file permissions
- ✓ Test in a staging environment
- ✓ Monitor logs and outputs
- ✓ Have a rollback plan
- ✓ Document how to use the tool



11. Key Takeaways

- ✓ Combine configuration, files, subprocess, APIs, logging and error handling
- ✓ Use environment variables and never hardcode secrets
- ✓ Handle failures gracefully and provide useful logs
- ✓ Use proper exit codes
- ✓ Make your scripts idempotent and safe to run multiple times
- ✓ Think about monitoring, alerts and rollback
- ✓ Follow best practices for production-ready automation

12. Common Beginner Mistakes

- ✗ Hardcoding passwords or API keys
- ✗ Not handling command or API failures
- ✗ Ignoring timeouts
- ✗ Not checking exit codes
- ✗ Writing logs only with print()
- ✗ Not validating user inputs
- ✗ Running scripts multiple times without locking
- ✗ Not testing error scenarios
- ✗ No documentation or rollback plan



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta